

Tabla de Contenidos

Introducción	1
Objetivos del Documento.....	...
Diseño SOA	2
Database	6
Backend	7
Maquetado	8
Procedimiento Almacenado	9
Servicios	9

Introducción

Portal Ristorino – Backend & Arquitectura SOA

Proyecto integrador académico – Diseño Avanzado de Software

Portal Ristorino es una plataforma web orientada a la búsqueda, recomendación y reserva de experiencias gastronómicas, integrando múltiples sistemas de restaurantes mediante una arquitectura orientada a servicios.

El sistema incorpora inteligencia artificial para interpretar búsquedas en lenguaje natural y generar contenido promocional dinámico en múltiples idiomas.

- Búsqueda de restaurantes con lenguaje natural
- Gestión de usuarios y autenticación
- Proceso de reservas
- Integración con servicios REST y SOAP
- Generación de contenido con IA (conceptual / mock)

El proyecto se desarrolla como caso de estudio académico, priorizando decisiones de diseño, arquitectura y escalabilidad.

Datos

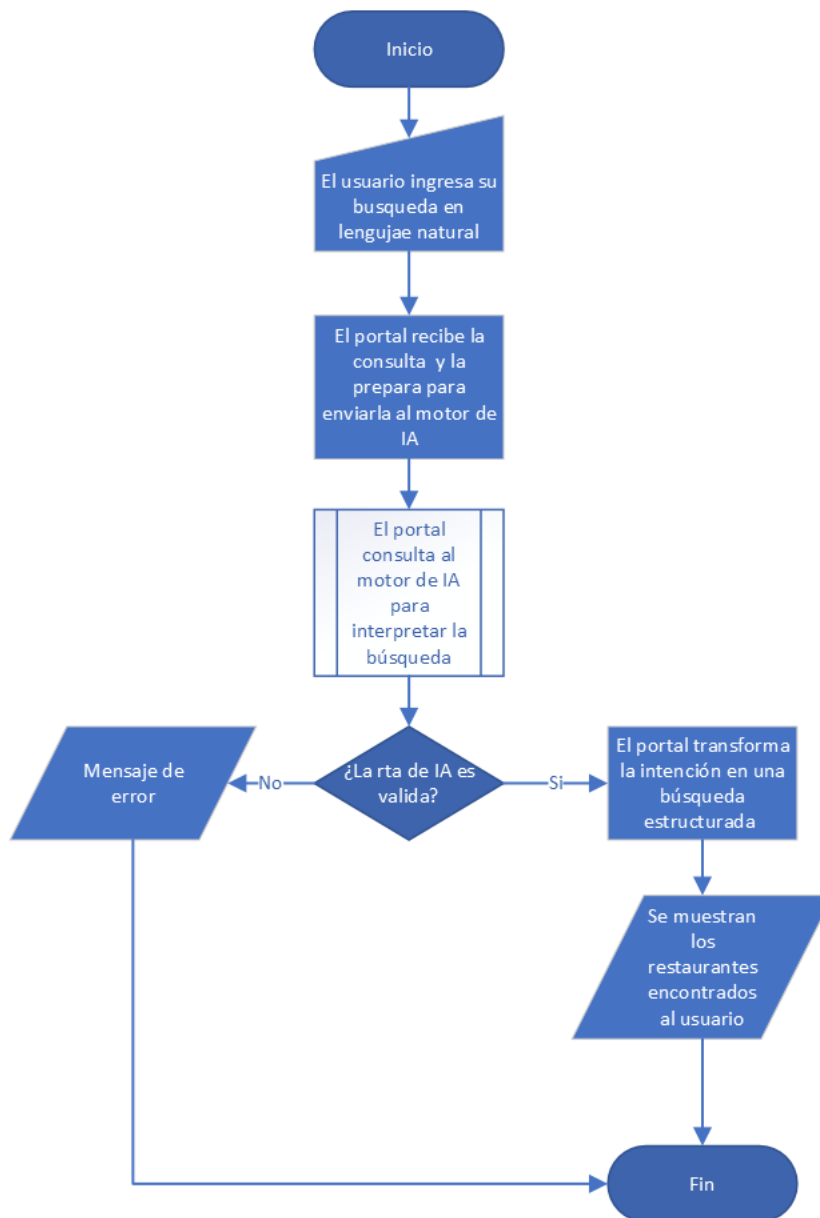
- Carrera: Ingeniería en Informática
- Tecnologías: Java (Spring Boot), REST, SOAP, SQL Server, Angular, IA
- Estado: En desarrollo



RISTORINO

Diseño SOA

Busqueda en lenguaje natural



PROCESO DE NEGOCIO N° 1

Nombre: Busqueda en lenguaje natural con IA.

Objetivos: Permitir que el usuario realice búsquedas de restaurantes utilizando lenguaje natural, interpretando su intención mediante un motor de inteligencia artificial que traduce dicha búsqueda en una consulta estructurada y precisa.

Alcance: Este proceso abarca desde el momento en que el usuario ingresa su búsqueda hasta la presentación de los resultados.

Incluye la interacción entre el portal y el motor de IA, pero **no contempla** el mantenimiento o entrenamiento del modelo de IA, que se maneja en otros procesos del sistema.

Descripción: Ver diagrama “Busqueda en lenguaje natural con IA”.

SERVICIOS

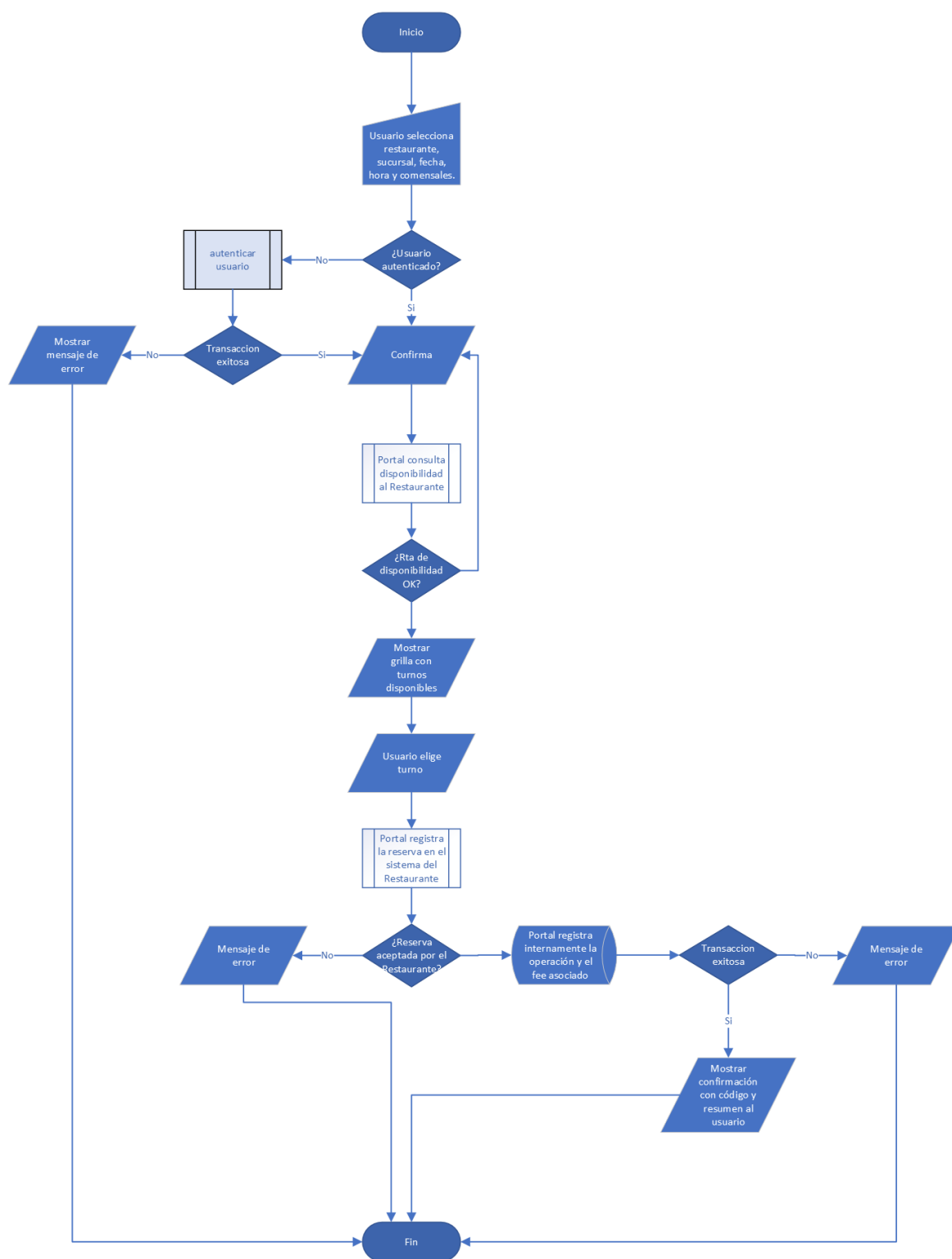
Identificador: 1

Descripción: Gestionar las búsquedas de restaurantes utilizando lenguaje natural.

Ubicación: Ristorino.

Operación: Permitir las búsquedas de restaurantes.				
Descripción: Permitir que el usuario realice búsquedas de restaurantes utilizando lenguaje natural.				
Solicitud				
Campo	Tipo	Tipo de Dato	Longitud [Min, Max]	Ejemplo de contenido del campo
Frase en lenguaje natural.	Mandatorio	String	1 a 20	restaurantes italianos cerca de mí.
Respuesta				
Campo	Tipo	Tipo de Dato	Longitud [Min, Max]	Ejemplo de contenido del campo
lista de restaurantes	Mandatorio	JSON		{ "restaurantes": [{ "nro_restaurante": 1, "razon_social": "La Trattoria", "localidad": "Córdoba", "provincia": "Córdoba" }] }

Reserva completa



PROCESO DE NEGOCIO N°2

Nombre: Reserva completa.

Objetivos: Permitir que un usuario autenticado realice la reserva de una mesa en un restaurante, seleccionando fecha, hora, sucursal y cantidad de comensales.

El sistema valida la disponibilidad con el restaurante, registra la reserva en su base de datos y devuelve un código de confirmación.

Alcance: Este proceso incluye la autenticación del usuario, la verificación de disponibilidad horaria con el restaurante, el registro de la reserva en el sistema y la generación del código de confirmación. No incluye la cancelación de reservas ni el procesamiento posterior de cobros o fees.

Descripción: Ver diagrama “Reserva completa”.

SERVICIOS

Identificador: 1

Descripción: Gestionar la reserva completa.

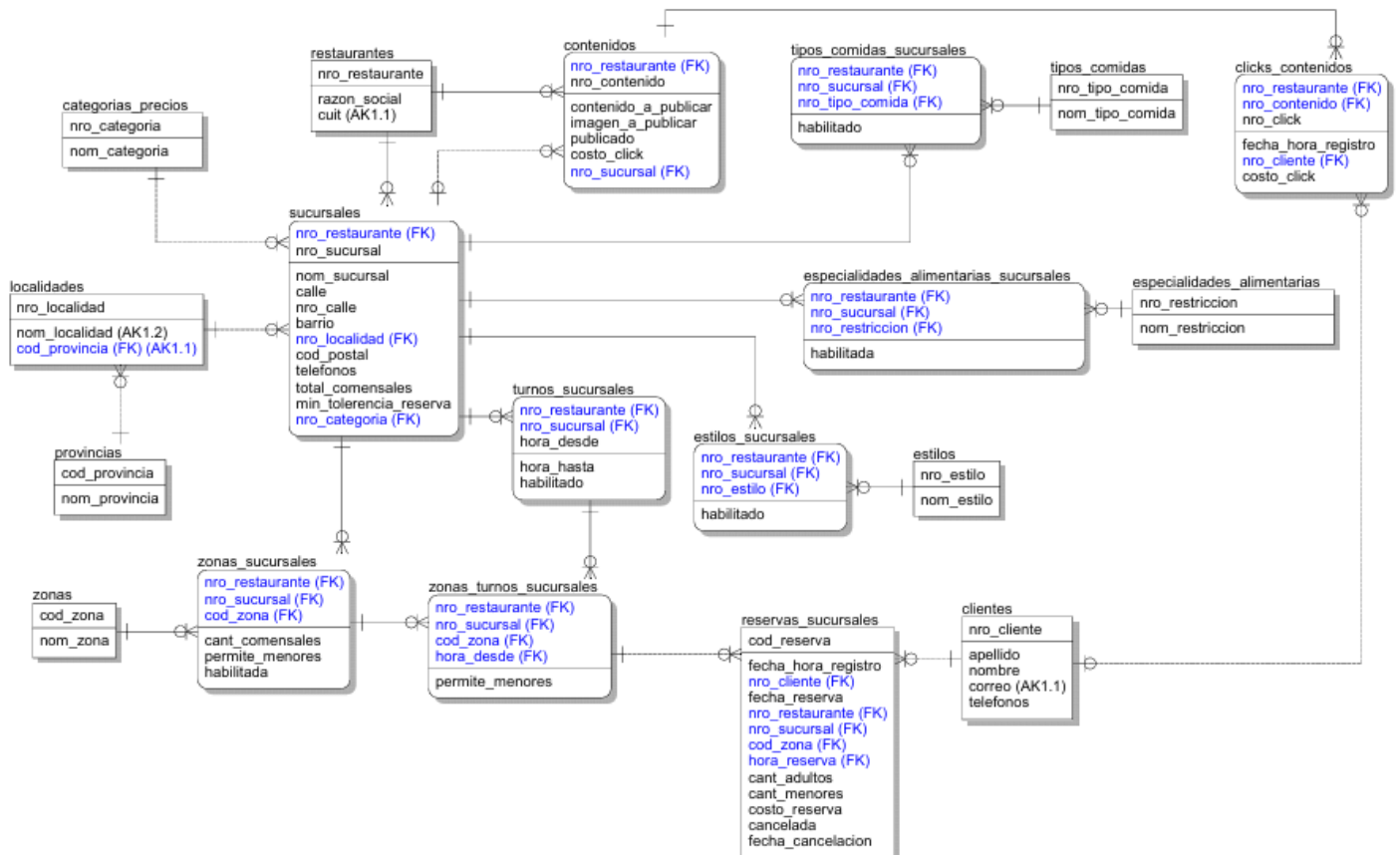
Ubicación: Ristorino.

Operación: Permitir realizar la reserva.				
Descripción: Permitir que un usuario realice la reserva de una mesa en un restaurante, seleccionando fecha, hora, sucursal y cantidad de comensales.				
Solicitud				
Campo	Tipo	Tipo de Dato	Longitud [Min, Max]	Ejemplo de contenido del campo
nro_cliente	Mandatorio	Numérico	[1,10]	1023
nro_restaurante	Mandatorio	Numérico	[1,10]	45
nro_sucursal	Mandatorio	Numérico	[1,10]	2
fecha_reserva	Mandatorio	Fecha	[10,10]	2025-11-20
hora_reserva	Mandatorio	Hora	[5,5]	21:30
cant_adultos	Mandatorio	Numérico	[1,2]	4
cant_menores	Opcional	Numérico	[1,2]	1
cod_zona	Opcional	Texto	[1,5]	“Arguello”

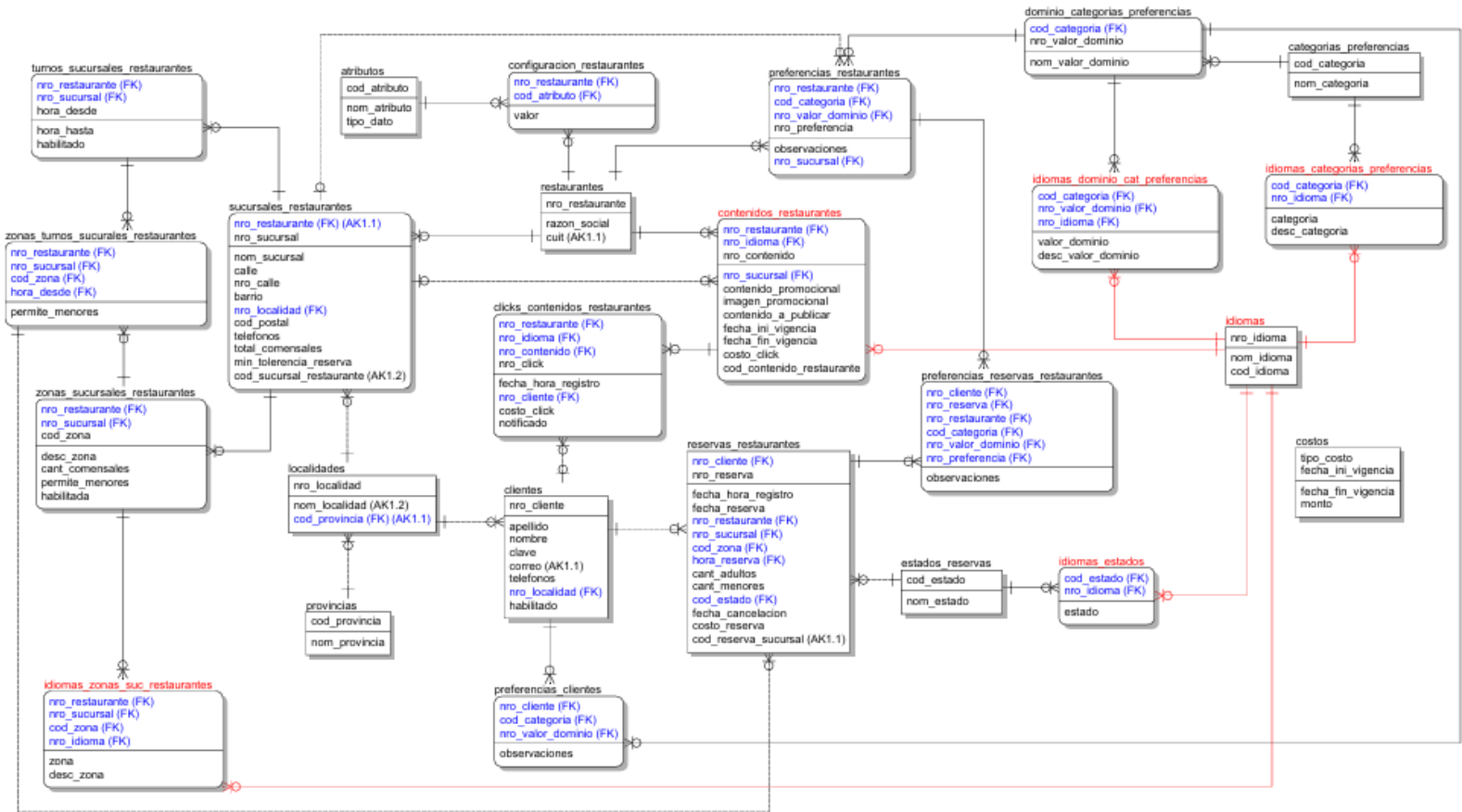
Respuesta				
Campo	Tipo	Tipo de Dato	Longitud [Min, Max]	Ejemplo de contenido del campo
cod_reserva_sucursal	Mandatorio	Texto	[6,10]	"R045-2025"
estado	Mandatorio	Texto	[1,20]	"Confirmada"
fecha_hora_registro	Mandatorio	Fecha/Hora	[19,19]	"2025-11-20 19:45:22"
monto_fee	Opcional	Decimal	[1,8]	120.50
mensaje	Opcional	Texto	[1,100]	"Reserva registrada correctamente".

Database

Restaurante



Ristorino – Portal



Backend

```

RistorinoPortalBackendApplication.java  ServletInitializer.java  RistorinoPortalBackendApplicationTests.java  application.properties

package ar.edu.ubp.das.ristorinoportalbackend;

import ...

@SpringBootApplication
public class RistorinoPortalBackendApplication {

    public static void main(String[] args) { SpringApplication.run(RistorinoPortalBackendApplication.class, args); }

}
    
```

Maquetado

Logo



El logotipo de Ristorino combina simplicidad y elegancia para reflejar la esencia del portal gastronómico.

La forma estilizada de una llama simboliza la pasión por la cocina y el arte culinario, mientras que el color violeta transmite sofisticación, creatividad y modernidad.

La tipografía en mayúsculas aporta claridad y equilibrio, reforzando la idea de un espacio digital confiable y atractivo para los amantes de la gastronomía.

Portal Ristorino



Procedimiento Almacenado

Contamos con varios procedimientos almacenados durante el desarrollo del proyecto, uno de los primeros:

```
-- =====
-- Procedimientos Almacenados
-- =====

-- Registrar los clics

CREATE PROCEDURE sp_registrar_click
    @nro_restaurante INT,
    @nro_idioma INT,
    @nro_contenido INT,
    @nro_cliente INT = NULL
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @nuevo_nro_click INT;

    SELECT @nuevo_nro_click = ISNULL(MAX(nro_click), 0) + 1
    FROM clicks_contenidos_restaurantes
    WHERE nro_restaurante = @nro_restaurante
        AND nro_idioma = @nro_idioma
        AND nro_contenido = @nro_contenido;

    DECLARE @costo_click DECIMAL(10,2);
    SELECT @costo_click = costo_click
    FROM contenidos_restaurantes
    WHERE nro_restaurante = @nro_restaurante
        AND nro_idioma = @nro_idioma
        AND nro_contenido = @nro_contenido;

    INSERT INTO clicks_contenidos_restaurantes
    (nro_restaurante, nro_idioma, nro_contenido, nro_click, fecha_hora_registro, nro_cliente, costo_click, notificado)
    VALUES
    (@nro_restaurante, @nro_idioma, @nro_contenido, @nuevo_nro_click, GETDATE(), @nro_cliente, @costo_click, 0);
END;
GO
```

Servicios

4 servicios para 4 procedimientos almacenados

1er servicio – Registro de clics

- **Rol del servicio** -> Registrar los clics de los usuarios (guardar esa acción en la base de datos).
- **Proveedor** -> Ristorino (quien ofrece el Endpoint).

- Solicitante -> El frontend del portal (el navegador del usuario o el cliente Angular que consume el servicio).

- Intermediario -> (todavía no aplica, mas adelante capaz un proxy o gateway).

Endpoint

Es la dirección física o lógica donde el servicio está disponible.

POST <http://localhost:8080/api/v1/clicks>

- Modelo del servicio

- Negocio -> Registrar clics está directamente vinculado con el proceso funcional del portal (relevamiento de promociones y cobro por clic).

Analisis

- Es un servicio REST de negocio.
- El recurso se llama clics (clicks en la URI).
- El verbo será POST porque estas creando un nuevo clic en la BD.
- El body será un JSON con los campos necesarios (los parámetros del procedimiento almacenado).
- El servidor (Spring Boot) será el proveedor del servicio.
- El cliente (Angular o Postman) será el solicitante.

Contrato

Servicio 1 – Registrar clic

Rol del servicio

Registrar los clics de los usuarios sobre contenidos promocionales, guardando esa acción en la base de datos definitiva mediante un procedimiento almacenado.

- **Proveedor:** Ristorino Portal Backend (Spring Boot)
- **Consumidor:** Frontend (Angular / Postman / navegador del usuario)
- **Intermediario:** no aplica en esta versión

Endpoint

POST <http://localhost:8080/api/v1/clicks>

Modelo del servicio

- **Tipo:** Servicio REST de negocio

- **Recurso:** clicks
- **Verbo HTTP:** POST (crea un nuevo registro de clic en la base de datos)
- **Proveedor:** servidor Spring Boot
- **Solicitante:** cliente Angular / Postman

Respuesta (Response)

Éxito

- Código: 201 Created
- Body (JSON): { "ok": true }

Errores

- 400 faltan campos o tipos inválidos
- 409 contenido o cliente inexistente
- 500 error al ejecutar el procedimiento almacenado o conexión a la BD.

2do servicio – Notificar clics pendientes

- Rol del servicio

Actualizar en la base de datos los clics que aún no fueron notificados, marcándolos como enviados y devolviendo un resumen con la cantidad total procesada.

El proceso puede ejecutarse manualmente desde un batch o endpoint, y forma parte del flujo de trazabilidad entre el portal y los restaurantes.

- **Proveedor:** Ristorino Portal Backend (Spring Boot)
- **Consumidor:** Proceso batch (Ejecutor / NotificarClicksJob) o cliente Postman
- **Intermediario:** no aplica en esta versión

Endpoint

POST http://localhost:8080/api/v1/clicks/notificar

- Modelo del servicio

- **Tipo:** Servicio REST técnico (batch de notificación)
- **Recurso:** clicks/notificar

- **Verbo HTTP:** POST (dispara la ejecución del procedimiento almacenado sp_notificar_clicks_pendientes)
- **Proveedor:** servidor Spring Boot
- **Solicitante:** batch del backend o cliente Postman

Respuesta (Response)

Éxito

- **Código:** 200 OK
- **Body (JSON):**
 - {
 - "ok": true,
 - "total_ok": 25,
 - "total_error": 0
 - }

Errores

- **500** error al ejecutar el procedimiento almacenado o conexión a la BD.
- **503** servicio temporalmente no disponible.

